[Request for proposal]

IMPLEMENTATION OF ETL TOOL

24.11.2025

EXUS Innovation

Table of Contents

1. Purpose and Background

EXUS requires the implementation of an ETL (Extract, Transform, Load) tool that automates the ingestion, transformation, and storage of structured data originating from text-based files. The tool will enable the integration of customer and financial product data from multiple systems into a standardized internal database schema.

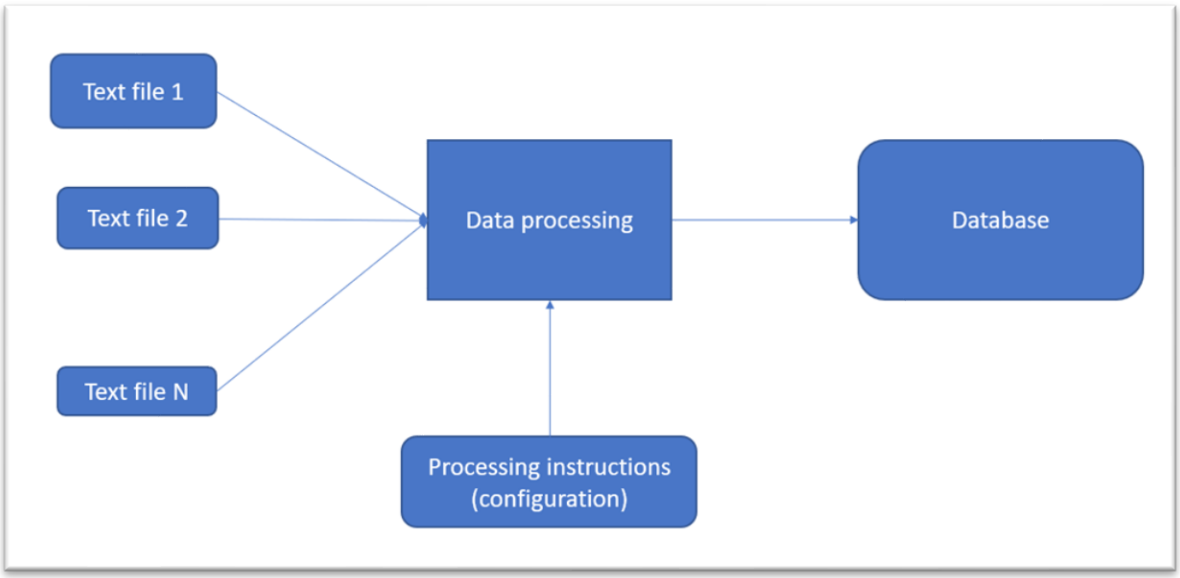
2. Project Objectives

- Automate ingestion of text-based customer and financial product data.
- Provide configuration-driven transformations without code customization.
- Ensure scalability, security, and maintainability of the ETL pipeline.
- Support flexible job scheduling and programmatic execution.

3. Scope of Work

The contractor will design, develop, and deliver a containerized ETL tool that reads input text files, transforms them according to configuration rules, and stores the processed data into a database schema. The database schema will contain 22 tables (approximately) covering information around accounts, customers, payments, and assets.

Below you can see a conceptual representation of the tool:



Notes:

1. Tool should support future schema extensions/updates (eg adding new columns or tables in the future)
2. ETL Tool will be a prototype with the intention to be used by technical or semi-technical users. We expect no UI for configuration/execution at this phase.

4. Functional Requirements

4.1. File Reading

The ETL tool must support configurable file ingestion through SFTP, handling both fixed-length and delimited formats (comma, pipe, semicolon, tab). It should recognize multiple encodings (ANSI, UTF-8, UTF-16) and handle headers, comments, and configurable line endings. Additional capabilities include skipping rows, filtering records, and managing 'bad record' thresholds.

1. Columns values can either be enclosed in special characters or not. Most common enclosing character is “” (e.g. “Text value”)
2. Support for file encoding ANSI, UTF-8, UTF-16 with or without BOM
3. Support for files with or without column headers
4. Ability to skip first N rows (configurable per file)
5. Ability to define line comment character per file (when line starts with this character, tool will ignore it)
6. Recognize \n, \r\n, \r for line end
7. Support for regex on date stamps for file names (eg filename_YYYDDMM hh:mm:ss.txt)
8. Configurable “bad” records threshold (when threshold reached, raise error for file)
9. Configurable empty file behavior (in case of empty either continue or raise error)
10. Ability to filter on specific rows based on column values (eg filter rows with Column1.value > 100)

4.2. Column Transformations

The tool must support a wide range of transformation functions such as type casting, trimming, regex, substring, null handling, default value substitution, and calculated fields using arithmetic and date functions. It should support function combinations and autogenerated keys.

- Ability to cast string to integer, decimal, boolean or date
- Parse multi-format dates based on ISO standards
- Ability to trim right, left or both
- Ability to get part of a column (substring)
- Ability to convert to uppercase/lowercase

- Ability to apply regex
- Ability to remove non-printable characters
- Ability to replace null/empty values with default values
- Ability for rounding
- Ability to define calculated columns based on existing ones. Calculations should support:
    - Fixed values
    - Basic arithmetic functions (+,-,*,?)
    - Date functions (extract day, day of week, month, year out of date column)
    - If then else statements
    - Concatenation of columns
    - Autogenerated keys based (UUID or sequence)
    - Combinations of functions and transformations (eg if cast(trim(column1, integer) > 5) then '0' else column2))

4.3. File Processing and Aggregation

The solution must allow merging multiple files, grouping, and aggregating data (min, max, sum, avg, count, etc.) with consistent transformation behavior across derived datasets. Also, solution should have the ability for test run on any file so that user can verify the transformations being made before actual execution.

- Ability to merge multiple files into one
- Ability to aggregate info out of a file (group by and aggregated columns)
- Aggregation functions:
    - Min
    - Max
    - First
    - Last
    - Count
    - Sum
    - Avg
- Aggregated files should have the same column transformation properties as the native ones (the ones applicable)

4.4. Database Schema Integration

Data will be copied into a database schema with configurable mappings, update behaviors, and delete capabilities.

- Ability to copy data from files to database tables
- Ability to define copy mode (insert/update, update only)
- Ability to define update behavior in case of empty source (update field or leave as is)

- Ability to delete data from schema

4.5. Job Orchestration

The ETL tool must support the definition and scheduling of jobs that execute data tasks in sequence. Integration with Apache Airflow or a built-in scheduler is preferred.

- Ability for a task to wait till file appears in folder
- Ability to raise error or continue in case of missing file.
- Ability for task to execute sql code in database (eg after copying info, update a flag)

5. Non-Functional Requirements

- Solution must be **container-based**, designed for **scalability** and **compatibility with Kubernetes** for orchestration and workload management.
- Development and version control will take place within **EXUS's GitHub organization**, following EXUS's internal contribution and review policies.
- All **source code, documentation, and build assets** (e.g., Dockerfiles, Helm charts, configuration files) will be **fully owned by EXUS** upon delivery.
- The solution should follow **best practices for containerization**, including stateless design and configuration through environment variables or Kubernetes config maps.
- The solution must be **fully compliant with EXUS's security baseline**, including code security, dependency management, secret handling, and infrastructure hardening practices.
- Deliverables must include all necessary **technical and operational documentation** to support deployment, maintenance, and scaling.
- API or scheduler-based job execution with retry and alerting support
- Functional requirements should be covered as much as possible through configuration files without the need to customize code per installation.
- Nice to have: tools/utilities that can boost configuration (eg automated parsing of source files and creating configuration etc). In general, anything that can speedup configuration of tool.
- Robust logging, monitoring, and error management
- Fully open-source technology stack. Note: EXUS current experience is around Python, sql and Airflow.
- Capability to process large text files (up to 100.000.000 records per file) in a matter of minutes (not hours).

6. Deliverables

- ETL tool source code and configuration files.
- Docker and Helm deployment artifacts.
- Technical, Operational and Deployment documentation.
- Automated tests (unit and end-to-end)
- Test and performance reports.

7. Vendor Response Guidelines

Proposals must include:

- Company profile, size and experience in similar projects
- Implementation phases and duration.
- Cost estimation with task breakdown
- High-level design and architecture overview.
- Explanation of how the proposed solution handles the sample use case (see appendix below).
- Post-delivery support and maintenance offering.

8. Evaluation Criteria

Proposals will be evaluated based on the following criteria:

- Technical Fit
- Cost
- Implementation duration
- Vendor experience in similar projects

9. Intellectual Property and Ownership

All deliverables, including source code, configuration files, and documentation, produced under this project shall remain the sole property of EXUS. EXUS will retain perpetual rights for use, modification, and redistribution.

10. Submission Instructions

Vendors must submit their proposals via email to **krap@exus.co.uk**.

The RFP process has the following phases:

Phase	Dates	Comments
RFP release	Nov 24, 2025	
Questions of Clarifications from Vendors	Nov 25 – Dec 5, 2025	Through email or meeting
Proposals submission	Nov 25 – Dec 19, 2025	
Proposals evaluation	Dec 22, 2025 – Jan 9, 2026	On Jan 9, Vendors will be informed whether they are shortlisted or not.
Shortlisted vendors evaluation	Jan 12, 2026 – Jan 16, 2026	Expect meetings for clarifications
Vendor selection	Jan 19 – Jan 23, 2026	

11. Appendix

11.1. Proposed Solution

Note: Proposed solution presented below is indicative. You may follow same approach or suggest you own

Proposed solution contains the following components:

11.1.1. Source file

This is a component that represents any source text file. Component contains metadata that describe file properties and columns. Also, it manages all behaviors described in the file reading and column transformation sections.

11.1.2. UnionDataset

This is a component that represents the union of multiple source files. User can define the columns and the mappings of each column to the source files. Also, user may define union or union all behavior.

11.1.3. AggregatedDataset

This is a component that represents merging/aggregation of info contained in source file. Source file can be either a File or Union Dataset. AggregatedDataset is used to extract aggregated info from the source. Users can define the source file, the columns, and the aggregation function for each (group by, min, max etc).

11.1.4. CopyData

Copy Data component copies data from any kind of file component (source, union, aggregated) to database schema. Users can define:

- the mapping of source and destination columns
- The matching key for the update
- the update behavior (in case of source empty)
- The copy mode (insert/Update or update only)

11.1.5. DeleteData

Delete Data component deletes data from the database schema. Users can define which data to be deleted (tables, rows).

11.1.6. Job

A job is a collection of tasks executed with specific order. Task that can be included in a job are either copydata or deletedata.

11.2. Example use case

Below you can see a sample use case for the etl-tool and, also, the way the proposed solution would support it:

11.2.1. Scenario

Assume that we have 2 systems that hold information about customers and their acquired products.

- SystemA contains products of productfamilyA and
- SystemB contains products of productfamilyB.

Both systems produce a file containing information about each acquired product together with customer details and associated address and phone number
SystemA produces the file at 1.00am and SystemB produces the file at 3.am.

File columns are presented below:

- RecordId

- CustomerId
- CustomerName
- CustomerVAT
- ProductCode
- ProductName
- Address
- Phonenumber

Information needs to be copied to the destination database. Database contains different tables to host information. The relevant tables are the following:
Customer

- CustomerId
- CustomerVAT
- CustomerName

Product

- ProductId
- CustomerId
- ProductName

Address

- AddressId
- CustomerId
- Address

Phone

- PhoneId
- CustomerId
- Phonenumber

11.2.2. Solution

Using the solution that we have developed, we need to copy the source files to the destination database. Since files are ready in different timeslots, we will need to build 2 jobs that will run for each System in different schedules.
In our example, we are going to showcase SystemA, since SystemB will follow the same configuration.
Based on the proposed solution above, the components that need to be created are the following:

11.2.2.1. Source files

We need one source file to describe the file exported from SystemA. We will need to define columns, datatypes, possible re-formats or casts etc.
Name of source file: SourceFileA

11.2.2.2. Union Datasets

Since source files come in different timeslots, we do not need to create any union datasets to combine information from both systems.

11.2.2.3. Aggregated dataset

We will need to create aggregateddatasets in order to extract customer, address and phone information.

Aggregated dataset 1: Customer
Source file: SourceFileA
Aggregated fields: CustomerName, CustomerVAT (aggregation function for each must be defined (first, last etc))
Group by fields: CustomerId

Aggregated dataset2: Address
Source file: SourceFileA
Aggregated fields: N/A
Group by fields: CustomerId, Address

Aggregated dataset3: Phone
Source file: SourceFileA
Aggregated fields: N/A
Group by fields: CustomerId, Phone

11.2.2.4. CopyData

We need to copy data from sources to destination tables. For each copy components, we need to define, the join and the update behavior of each column based on the functional requirements stated in the relative section.

CopyData1: Accounts
Source: SourceFileA
Destination: Product
Record Join: RecordId -> ProductId
Mappings (Source-> Destination)

- RecordId -> ProductId
- CustomerId -> CustomerId
- ProductName -> ProductName

CopyData2: Customers
Source: AggregatedDataset Customer
Destination: Customer
Record Join: CustomerId-> CustomerId
Mappings (Source-> Destination)

- CustomerId -> CustomerId
- CustomerVAT -> CustomerVAT
- CustomerName -> CustomerName

CopyData3: Addresses
Source: AggregatedDataset Address
Destination: Address

Record Join: CustomerId, address-> CustomerId, Address
Mappings (Source-> Destination)

- Autoinc function -> AddressId
- CustomerId -> CustomerId
- Address -> Address

CopyData4: Phones
Source: AggregatedDataset Phone
Destination: Phone
Record Join: CustomerId, phonumber-> CustomerId, phonenumber
Mappings (Source-> Destination)

- Autoinc function -> PhoneId
- CustomerId -> CustomerId
- Phonenumber-> Phonenumber

11.2.2.5. Job

We need create a job in order to copy data to destination tables.

Job1: SystemA

Tasks:

1. Wait for SourceFileA
2. CopyData1
3. CopyData2
4. CopyData3
5. CopyData4